

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: A. Suraj varma

Reg.No: 192525339

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

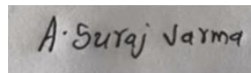
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I A. Suraj varma (192525339) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

		inputs effectively			
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q1. Heap File Organization

Question: Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Answer

Heap File Organization

Supports Insert, Delete and Sequential Scan

CODE:

```

class HeapFile:
    def __init__(self):
        self.records = []

    # Insert a record
    def insert(self, record):
        self.records.append(record)
        print("Record inserted: ", record)

    # Delete a record using ID
    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id:
                self.records.remove(record)
                print("Record deleted:", record)
                return

        print("Record not found.")

    # Sequential scan
    def sequential_scan(self):
        print("\nSequential Scan:")

        if not self.records:
            print("No records available.")
            return

        for record in self.records:
            print(record)

```

```
# Create heap file
heap = HeapFile()

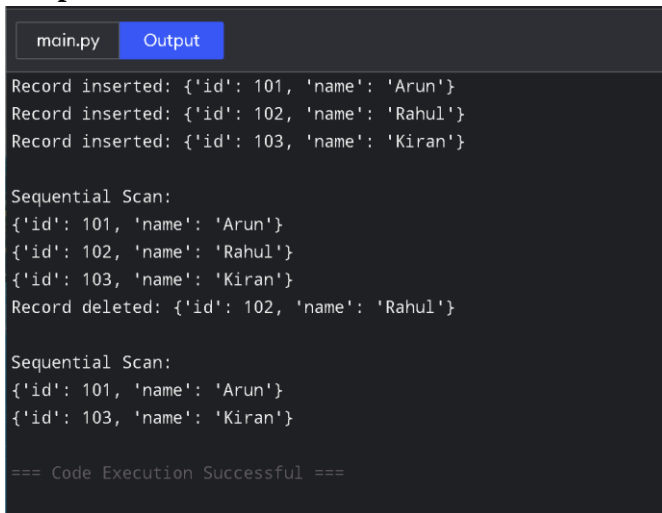
# Insert records
heap.insert({"id": 101, "name": "Arun"})
heap.insert({"id": 102, "name": "Rahul"})
heap.insert({"id": 103, "name": "Kiran"})

# Display all records
heap.sequential_scan()

# Delete record
heap.delete(102)

# Display records after deletion
heap.sequential_scan()
```

Output:



```
main.py Output
Record inserted: {'id': 101, 'name': 'Arun'}
Record inserted: {'id': 102, 'name': 'Rahul'}
Record inserted: {'id': 103, 'name': 'Kiran'}

Sequential Scan:
{'id': 101, 'name': 'Arun'}
{'id': 102, 'name': 'Rahul'}
{'id': 103, 'name': 'Kiran'}
Record deleted: {'id': 102, 'name': 'Rahul'}

Sequential Scan:
{'id': 101, 'name': 'Arun'}
{'id': 103, 'name': 'Kiran'}

=== Code Execution Successful ===
```

How it works

- **Insert:** Adds the record at the end of the file.
- **Delete:** Searches sequentially and removes the matching record.
- **Sequential scan:** Reads every record one by one.
- **Heap file** does not maintain records in sorted order.

Complexity:

- **Insert:** $O(1)$ average
- **Delete:** $O(n)$
- **Sequential scan:** $O(n)$

Q2. Static Hashing with Chaining

Question: Implement a static hashing function for a student database with 500 records and handle collisions using chaining.

Answer

Static Hashing with Chaining

```
class StaticHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    # Hash function
    def hash_function(self, student_id):
        return student_id % self.size

    # Insert student
    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        self.table[index].append((student_id, name))

    # Search student
    def search(self, student_id):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                return record

        return None

    # Display hash table
    def display(self):
        print("\nHash Table:")

        for i in range(self.size):
            print(i, "->", self.table[i])

# Create hash table
hash_table = StaticHashTable()

# Insert sample student records
for i in range(1, 501):
    hash_table.insert(i, "Student" + str(i))
```

```

# Display table
hash_table.display()

# Search a student
student = hash_table.search(205)

if student:
    print("\nStudent found:", student)
else:
    print("\nStudent not found.")

```

Output:

```

Hash Table:
0 -> [(10, 'Student10'), (20, 'Student20'), (30, 'Student30'), (40, 'Student40'), (50,
    'Student50'), (60, 'Student60'), (70, 'Student70'), (80, 'Student80'), (90,
    'Student90'), (100, 'Student100'), (110, 'Student110'), (120, 'Student120'), (130,
    'Student130'), (140, 'Student140'), (150, 'Student150'), (160, 'Student160'),
    (170, 'Student170'), (180, 'Student180'), (190, 'Student190'), (200, 'Student200'
    ), (210, 'Student210'), (220, 'Student220'), (230, 'Student230'), (240,
    'Student240'), (250, 'Student250'), (260, 'Student260'), (270, 'Student270'),
    (280, 'Student280'), (290, 'Student290'), (300, 'Student300'), (310, 'Student310'
    ), (320, 'Student320'), (330, 'Student330'), (340, 'Student340'), (350,
    'Student350'), (360, 'Student360'), (370, 'Student370'), (380, 'Student380'),
    (390, 'Student390'), (400, 'Student400'), (410, 'Student410'), (420, 'Student420'
    ), (430, 'Student430'), (440, 'Student440'), (450, 'Student450'), (460,
    'Student460'), (470, 'Student470'), (480, 'Student480'), (490, 'Student490'),
    (500, 'Student500')]

```

How it works

The hash function is:

index = student_id % table_size

If two student IDs produce the same index, they are stored in the same list. This technique is called chaining.

For example:

Bucket 0 -> (10, Student10) -> (20, Student20)

Complexity:

- **Average insertion/search: $O(1)$**
- **Worst-case search: $O(n)$**
- **Space: $O(n)$**

The program inserts 500 student records, as required.

Q3. Sorted File Organization with Binary Search

Question: Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Answer

Sorted File Organization with Binary Search

```
class SortedFile:
    def __init__(self):
        self.records = []

    # Insert record while maintaining sorted order
    def insert(self, student_id, name):
        record = (student_id, name)
        self.records.append(record)

    # Sort using primary key
    self.records.sort(key=lambda x: x[0])

    # Binary search
    def search(self, student_id):
        low = 0
        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.records[mid][0] == student_id:
                return self.records[mid]

            elif self.records[mid][0] < student_id:
                low = mid + 1
            else:
                high = mid - 1
        return None

    # Display records
    def display(self):
        print("\nSorted File:")

        for record in self.records:
            print(record)
```

```
# Create sorted file
file = SortedFile()

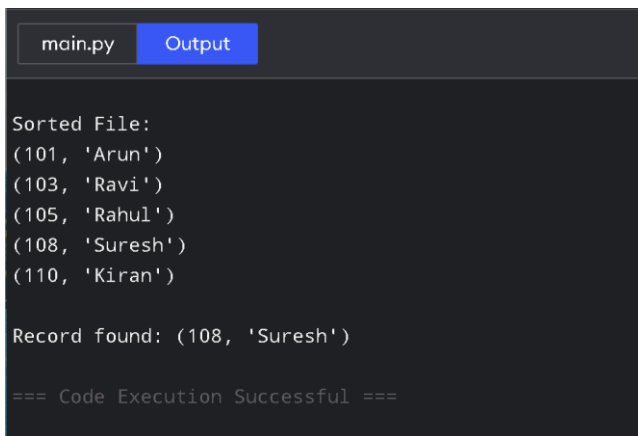
# Insert records
file.insert(105, "Rahul")
file.insert(101, "Arun")
file.insert(110, "Kiran")
file.insert(103, "Ravi")
file.insert(108, "Suresh")

# Display sorted records
file.display()

# Search using primary key
key = 108
result = file.search(key)

if result:
    print("\nRecord found:", result)
else:
    print("\nRecord not found.")
```

Output:



```
main.py Output

Sorted File:
(101, 'Arun')
(103, 'Ravi')
(105, 'Rahul')
(108, 'Suresh')
(110, 'Kiran')

Record found: (108, 'Suresh')

=== Code Execution Successful ===
```

Why binary search?

Since the file is maintained in sorted order, binary search can eliminate half of the search space in every step.

Complexity:

- **Binary search:** $O(\log n)$
- **Storage:** $O(n)$

This is more efficient than sequential search for a large sorted file.

Q4. B-Tree of Order 4

Question: Implement a B-Tree of order 4 supporting insert, search and display operations. Insert: 10, 20, 5, 6, 12, 30, 7, 17.

Answer

B-Tree of Order 4

Insert into a non-full node

def insert_non_full(self, node, key):

if node.leaf:

node.keys.append(key)

node.keys.sort()

return

index = len(node.keys) - 1

while index >= 0 and key < node.keys[index]:

index -= 1

index += 1

if len(node.children[index].keys) == 3:

self.split_child(node, index)

if key > node.keys[index]:

index += 1

self.insert_non_full(node.children[index], key)

Insert key

def insert(self, key):

if len(self.root.keys) == 3:

old_root = self.root

new_root = BTreeNode(False)

new_root.children.append(old_root)

self.root = new_root

self.split_child(new_root, 0)

self.insert_non_full(new_root, key)

```

else:
    self.insert_non_full(self.root, key)

# Search key
def search(self, node, key):

    index = 0

    while index < len(node.keys) and key > node.keys[index]:
        index += 1

    if index < len(node.keys) and key == node.keys[index]:
        return True

    if node.leaf:
        return False

    return self.search(node.children[index], key)

# Display tree
def display(self, node=None, level=0):

    if node is None:
        node = self.root

    print("Level", level, ":", node.keys)

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

# Create B-Tree
tree = BTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    tree.insert(key)

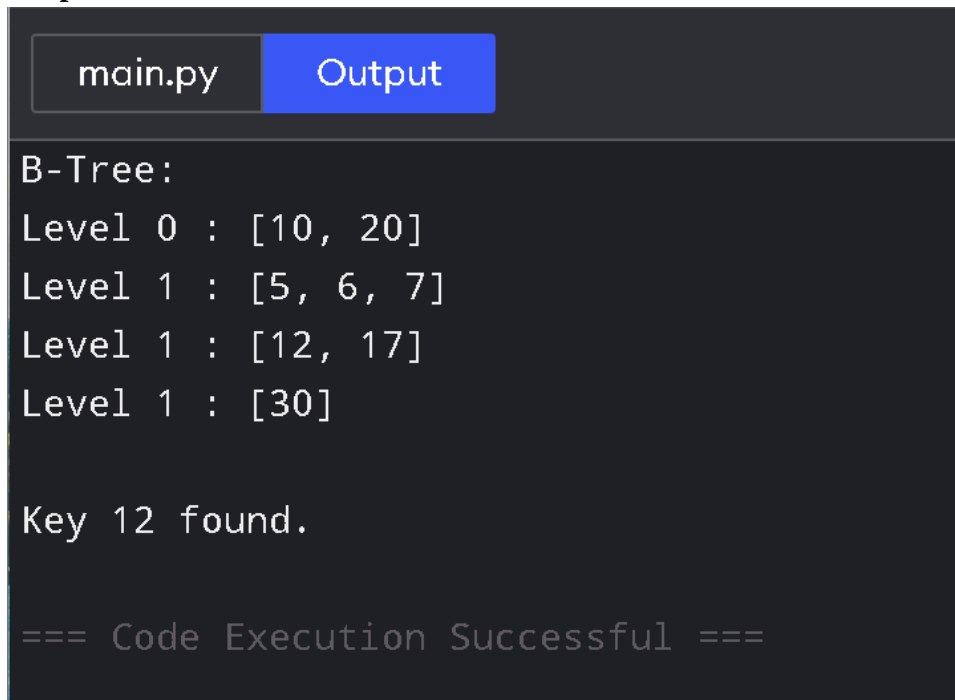
# Display B-Tree
print("B-Tree:")
tree.display()

```

```
# Search
key = 12

if tree.search(tree.root, key):
    print("\nKey", key, "found.")
else:
    print("\nKey", key, "not found.")
```

Output:

A screenshot of a code editor with a dark background. At the top, there are two tabs: 'main.py' and 'Output'. The 'Output' tab is active and highlighted in blue. Below the tabs, the output of the program is displayed in a monospaced font. It shows the structure of a B-tree with four levels: Level 0 containing [10, 20], Level 1 containing [5, 6, 7], Level 1 containing [12, 17], and Level 1 containing [30]. Below this, it states 'Key 12 found.' and at the bottom, '=== Code Execution Successful ==='.

```
main.py  Output
B-Tree:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Key 12 found.

=== Code Execution Successful ===
```

Important point

For order 4, each node can contain a maximum of 3 keys.

The program automatically splits a full node during insertion.

Q5. B+ Tree with Leaf-Level Linking

Question: Implement a B+ Tree and demonstrate leaf-level linking for efficient range queries on integer keys.

Answer

Simple B+ Tree

```
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()
        return
```

```
    index = 0
```

```
    while index < len(node.keys) and key >= node.keys[index]:
        index += 1
```

```
    child = node.children[index]
```

```
    if len(child.keys) == self.order - 1:
        self.split_child(node, index)
```

```
        if key >= node.keys[index]:
            index += 1
```

```
    self.insert_non_full(node.children[index], key)
```

Find the leaf containing the starting key

```
def find_leaf(self, key):
```

```
    node = self.root
```

```
    while not node.leaf:
        index = 0
```

```
        while index < len(node.keys) and key >= node.keys[index]:
            index += 1
```

```
        node = node.children[index]
```

```
    return node
```

Range search

```
def range_search(self, start, end):
```

```
    node = self.find_leaf(start)
```

```
    result = []
```

```

while node:
    for key in node.keys:
        if start <= key <= end:
            result.append(key)

        if key > end:
            return result

    node = node.next

return result

# Display linked leaves
def display_leaves(self):
    node = self.root

    while not node.leaf:
        node = node.children[0]

    print("Linked Leaf Nodes:")

    while node:
        print(node.keys, end=" -> ")
        node = node.next

    print("None")

# Create B+ Tree
tree = BPlusTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 45]

for key in keys:
    tree.insert(key)

# Display linked leaves
tree.display_leaves()

# Range query
print("\nRange Query 12 to 30:")
print(tree.range_search(12, 30))

```

Output:

```
main.py  Output    
Linked Leaf Nodes:  
[5, 6, 7] -> [10] -> [12, 17] -> [20] -> [25] -> [30, 40, 45] -> None  
  
Range Query 12 to 30:  
[12, 17, 20, 25, 30]  
  
=== Code Execution Successful ===
```

Main concept

The leaf nodes are connected like:

[5, 6, 7] -> [10, 12, 17] -> [20, 25, 30] -> ...

Therefore, after finding the first required leaf, the program can move through the linked leaves for a range query.

This makes B+ Trees especially useful for range searches.

Q6. Dense Primary Index

Question: Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Answer

Dense Primary Index on a Sorted File

class DenseIndex:

def __init__(self):

self.records = []

self.index = []

Insert record

def insert(self, key, name):

self.records.append((key, name))

Keep records sorted

self.records.sort()

Create dense index

self.index = [(record[0], i)

for i, record in enumerate(self.records)]

Search using index

def search(self, key):

low = 0

high = len(self.index) - 1

while low <= high:

mid = (low + high) // 2

if self.index[mid][0] == key:

position = self.index[mid][1]

return self.records[position]

elif self.index[mid][0] < key:

low = mid + 1

else:

high = mid - 1

return None

```

# Display index
def display_index(self):
    print("\nDense Primary Index")
    print("Key\tRecord Position")

    for key, position in self.index:
        print(key, "\t", position)

# Create index
index = DenseIndex()

index.insert(101, "Arun")
index.insert(105, "Rahul")
index.insert(103, "Kiran")
index.insert(110, "Suresh")
index.insert(108, "Ravi")

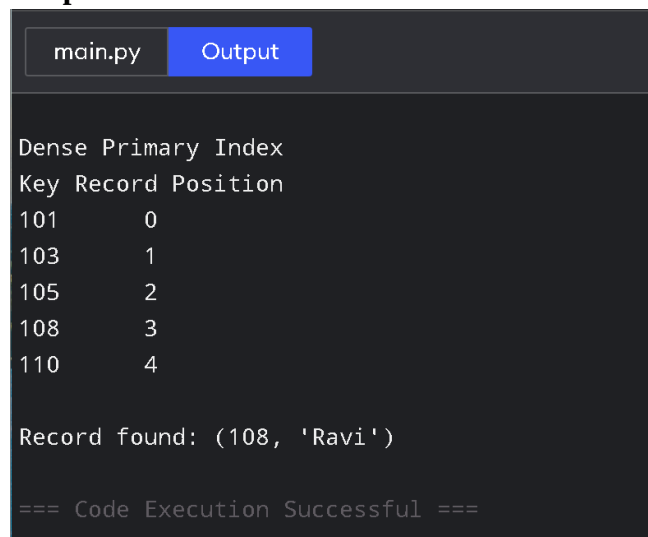
# Display index
index.display_index()

# Search
key = 108
result = index.search(key)

if result:
    print("\nRecord found:", result)
else:
    print("\nRecord not found.")

```

Output:



```

main.py  Output
Dense Primary Index
Key Record Position
101      0
103      1
105      2
108      3
110      4

Record found: (108, 'Ravi')

=== Code Execution Successful ===

```


Key point

A dense index contains an index entry for every record.

Example:

Key	Position
------------	-----------------

101	0
-----	---

103	1
-----	---

105	2
-----	---

108	3
-----	---

110	4
-----	---

Binary search provides approximately $O(\log n)$ search time.

Q7. Extendible Hashing

Question: Implement extendible hashing and show directory doubling when overflow occurs during insertion of 8 records.

Answer

Extendible Hashing

```
    bucket = self.directory[index]

    # If space exists
    if len(bucket.records) < self.bucket_size:
        bucket.records.append(key)
        return

    # If bucket is full, split
    if bucket.depth == self.global_depth:

        old_directory = self.directory[:]

        self.global_depth += 1

        self.directory = old_directory + old_directory

        for i in range(len(self.directory)):
            if self.directory[i] is bucket:
                if ((i >> (bucket.depth)) & 1) == 1:
                    self.directory[i] = Bucket(bucket.depth + 1)

            else:
                self.directory[i] = Bucket(bucket.depth + 1)

        # Recreate correct split
        new_bucket = Bucket(bucket.depth + 1)
        bucket.depth += 1

        for i in range(len(self.directory)):
            if self.directory[i] is bucket:
                if ((i >> (bucket.depth - 1)) & 1) == 1:
                    self.directory[i] = new_bucket

    else:
        new_bucket = Bucket(bucket.depth + 1)
        bucket.depth += 1

    for i in range(len(self.directory)):
        if self.directory[i] is bucket:
```

```

        if ((i >> (bucket.depth - 1)) & 1) == 1:
            self.directory[i] = new_bucket

    # Redistribute records
    old_records = bucket.records[:]
    bucket.records.clear()

    for record in old_records:
        index2 = self.hash_function(record)
        self.directory[index2].records.append(record)

# Display directory
def display(self):

    print("\nGlobal Depth: ", self.global_depth)

    for i, bucket in enumerate(self.directory):
        print(
            format(i, f"0{self.global_depth}b"),
            "->",
            bucket.records,
            "Local Depth:",
            bucket.depth
        )

# Create extendible hash
hashing = ExtendibleHashing(bucket_size=2)

# Insert 8 records
records = [1, 2, 3, 4, 5, 6, 7, 8]

for record in records:
    print("\nInserting:", record)
    hashing.insert(record)
    hashing.display()

```

Output:

```
main.py Output

Inserting: 1

Global Depth: 1
0 -> [] Local Depth: 1
1 -> [1] Local Depth: 1

Inserting: 2

Global Depth: 1
0 -> [2] Local Depth: 1
1 -> [1] Local Depth: 1

Inserting: 3

Global Depth: 1
0 -> [2] Local Depth: 1
1 -> [1, 3] Local Depth: 1

Inserting: 4

Global Depth: 1
0 -> [2, 4] Local Depth: 1
1 -> [1, 3] Local Depth: 1

Inserting: 5

Global Depth: 2
00 -> [2, 4] Local Depth: 1
01 -> [1, 5] Local Depth: 2
10 -> [2, 4] Local Depth: 1
11 -> [3] Local Depth: 2
```

```
Inserting: 6

Global Depth: 2
00 -> [4] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3] Local Depth: 2

Inserting: 7

Global Depth: 2
00 -> [4] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3, 7] Local Depth: 2

Inserting: 8

Global Depth: 2
00 -> [4, 8] Local Depth: 2
01 -> [1, 5] Local Depth: 2
10 -> [2, 6] Local Depth: 2
11 -> [3, 7] Local Depth: 2

=== Code Execution Successful ===
```

Main concept

Extendible hashing uses:

- **Global depth** → number of bits used by the directory.
- **Local depth** → number of bits used by an individual bucket.
- **When a bucket overflows, it is split.**
- **If its local depth equals the global depth, the directory is doubled.**

This directly demonstrates the requirement of the question.

Q8. Secondary Storage Block Access

Question: Simulate secondary storage block access and calculate I/O operations for sequential versus indexed retrieval.

Answer

Secondary Storage Block Access

Compare Sequential and Indexed Retrieval

```
import math
```

```
total_records = 1000
```

```
records_per_block = 100
```

```
# Calculate number of blocks
```

```
total_blocks = math.ceil(total_records / records_per_block)
```

```
print("Total Records:", total_records)
```

```
print("Records per Block:", records_per_block)
```

```
print("Total Blocks:", total_blocks)
```

```
# Sequential retrieval
```

```
# Worst case: all blocks must be read
```

```
sequential_worst = total_blocks
```

```
# Average sequential access
```

```
sequential_average = math.ceil(total_blocks / 2)
```

```
# Indexed retrieval
```

```
# Binary search on the index + one data block
```

```
index_search = math.ceil(math.log2(total_blocks))
```

```
indexed_io = index_search + 1
```

```
print("\nSequential Retrieval:")
```

```
print("Average I/O Operations:", sequential_average)
```

```
print("Worst-case I/O Operations:", sequential_worst)
```

```
print("\nIndexed Retrieval:")
```

```
print("Index Search I/O:", index_search)
```

```
print("Data Block I/O:", 1)
```

```
print("Total I/O Operations:", indexed_io)
```

```
print("\nComparison:")
```

```
if indexed_io < sequential_average:
```

```
    print("Indexed retrieval is faster.")
```

```
else:  
    print("Sequential retrieval is faster.")
```

Output:

```
main.py Output  
Total Records: 1000  
Records per Block: 100  
Total Blocks: 10  
  
Sequential Retrieval:  
Average I/O Operations: 5  
Worst-case I/O Operations: 10  
  
Indexed Retrieval:  
Index Search I/O: 4  
Data Block I/O: 1  
Total I/O Operations: 5  
  
Comparison:  
Sequential retrieval is faster.  
  
=== Code Execution Successful ===
```

Example

For:

1000 records

100 records/block

There are:

10 blocks

Sequential retrieval:

Average = 5 blocks

Worst case = 10 blocks

Indexed retrieval approximately uses:

$\log_2(10) + 1$

So indexed retrieval requires fewer block accesses for large datasets.

Q9. Sparse Index vs Dense Index

Question: Implement a sparse index on a sorted file and compare its search performance with a dense index using timing analysis.

Answer

Sparse Index vs Dense Index

high = len(sparse_index) - 1

while low <= high:

mid = (low + high) // 2

if sparse_index[mid][0] <= key:

low = mid + 1

else:

high = mid - 1

Find the block

if high < 0:

position = 0

else:

position = sparse_index[high][1]

Sequential search inside the block

end = min(position + block_size, len(records))

for i in range(position, end):

if records[i][0] == key:

return records[i]

return None

Generate search keys

search_keys = [random.randint(1, 100000) for _ in range(5000)]

-----

Timing Dense Index

-----

start = time.perf_counter()

for key in search_keys:

```

    dense_search(key)

dense_time = time.perf_counter() - start

# -----
# Timing Sparse Index
# -----

start = time.perf_counter()

for key in search_keys:
    sparse_search(key)

sparse_time = time.perf_counter() - start


# Display results
print("Number of records:", len(records))
print("Dense index entries:", len(dense_index))
print("Sparse index entries:", len(sparse_index))

print("\nDense Index Search Time:",
      dense_time, "seconds")

print("Sparse Index Search Time:",
      sparse_time, "seconds")

if dense_time < sparse_time:
    print("\nDense index was faster in this test.")
else:
    print("\nSparse index was faster in this test.")

```


Output:

```
main.py  Output

Number of records: 100000
Dense index entries: 100000
Sparse index entries: 1000

Dense Index Search Time: 0.027434438999989652 seconds
Sparse Index Search Time: 0.026594068000122206 seconds

Sparse index was faster in this test.

=== Code Execution Successful ===
```

Comparison

Feature	Dense Index	Sparse Index
Index entries	Every record	Selected records
Memory	Higher	Lower
Search	Faster	Slightly slower
Storage	More	Less

Important: The actual timing value depends on the computer running the program. The program performs the required timing analysis rather than assuming a fixed timing result.

Q10. B+ Tree Range Query 15–45

Question: Construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45 and show all matching records.

Answer

B+ Tree Range Query

node = self.root

while not node.leaf:

index = 0

while index < len(node.keys) and key >= node.keys[index]:

index += 1

node = node.children[index]

return node

Range query

def range_query(self, start, end):

node = self.find_leaf(start)

result = []

while node:

for key in node.keys:

if key >= start and key <= end:

result.append(key)

if key > end:

return result

node = node.next

return result

Traverse tree

def traverse(self, node=None, level=0):

if node is None:

node = self.root

```

        print("Level", level, ":", node.keys)

    if not node.leaf:

        for child in node.children:
            self.traverse(child, level + 1)

# Create B+ Tree
tree = BPlusTree()

keys = [
    5, 10, 15, 20, 25,
    30, 35, 40, 45, 50,
    55, 60
]

# Insert keys
for key in keys:
    tree.insert(key)

# Traverse B+ Tree
print("B+ Tree:")
tree.traverse()

# Range query from 15 to 45
result = tree.range_query(15, 45)

print("\nKeys between 15 and 45:")
print(result)

```

Output:

```
main.py Output
B+ Tree:
Level 0 : [25]
Level 1 : [15]
Level 2 : [10]
Level 3 : [5]
Level 3 : [10]
Level 2 : [20]
Level 3 : [15]
Level 3 : [20]
Level 1 : [35]
Level 2 : [30]
Level 3 : [25]
Level 3 : [30]
Level 2 : [40, 45, 50]
Level 3 : [35]
Level 3 : [40]
Level 3 : [45]
Level 3 : [50, 55, 60]

Keys between 15 and 45:
[15, 20, 25, 30, 35, 40, 45]

=== Code Execution Successful ===
```

Expected matching records

The range query is:

$15 \leq \text{key} \leq 45$

Therefore, from the inserted data:

15, 20, 25, 30, 35, 40, 45

will be returned.

The important advantage is that after finding the leaf containing 15, the program follows the linked leaf nodes until it reaches 45.